SESSION 18

1. **Create a SQL view named TopRatedRestaurants that selects the restaurant name, average rating, and total number of reviews from a table of Zomato-style restaurant reviews, showing only restaurants with an average rating above 4.0.**

```
CREATE VIEW TopRatedRestaurants AS
SELECT
   restaurant_name,
   AVG(rating) AS average_rating,
   COUNT(*) AS total_reviews
FROM RestaurantReviews
GROUP BY restaurant_name
HAVING AVG(rating) > 4.0;
```

Explanation:

- CREATE VIEW: Creates a view named TopRatedRestaurants.
- AVG(rating): Calculates the average rating of each restaurant.
- COUNT(*): Counts the total number of reviews for each restaurant.
- GROUP BY restaurant_name: Groups reviews by restaurant name.
- HAVING AVG(rating) > 4.0: Displays only restaurants with an average rating above 4.0.

Result: The view displays each qualifying restaurant's name, average rating, and total number of reviews.

**2. Update the TopRatedRestaurants view to also include the city column from the original restaurants table by joining the relevant tables.

Hint: Use an INNER JOIN to combine data from both tables in your view definition.**

```
CREATE OR REPLACE VIEW TopRatedRestaurants AS
```

```
SELECT
    r.restaurant_name,
    r.city,
    AVG(rv.rating) AS average_rating,
    COUNT(*) AS total_reviews
FROM Restaurants r
INNER JOIN RestaurantReviews rv
    ON r.restaurant_name = rv.restaurant_name
GROUP BY
    r.restaurant_name,
    r.city
HAVING AVG(rv.rating) > 4.0;
```

Explanation:

- CREATE OR REPLACE VIEW: Updates the existing view definition.
- INNER JOIN: Combines the Restaurants and RestaurantReviews tables using the restaurant name.
- r.city: Includes the city of each restaurant.
- AVG(rv.rating): Calculates the average rating.
- COUNT(*): Counts the total number of reviews.
- GROUP BY: Groups the results by restaurant name and city.
- HAVING AVG(rv.rating) > 4.0: Shows only restaurants with an average rating above 4.0.

Result: The updated view displays the restaurant name, city, average rating, and total number of reviews for restaurants rated above 4.0.

3. Try to update the average rating column directly through the TopRatedRestaurants view and observe what error or limitation occurs. Write down the exact error message and explain why this happens based on SQL view limitations.

```
UPDATE TopRatedRestaurants

SET average_rating = 4.5

WHERE restaurant_name = 'ABC Restaurant';
```

Error Message:

In MySQL, this operation typically produces an error similar to:

ERROR 1348 (HY000): Column 'average_rating' is not updatable

Explanation:

The TopRatedRestaurants view contains the AVG(rating) aggregate function and uses GROUP BY and HAVING. Such views are not updatable in MySQL because the average_rating column is a calculated value derived from multiple rows in the original RestaurantReviews table.

To change the average rating, the underlying review records must be updated instead, where appropriate. The average rating will then be recalculated when the view is queried.

**4. Create a view called DailyOrderSummary that shows, for each date, the total number of food orders and the total revenue from a Swiggy-style orders table. Ensure the view only includes dates from the last 30 days.

Constraint: Use WHERE and GROUP BY clauses in your view definition.**

```
CREATE VIEW DailyOrderSummary AS

SELECT

   DATE(order_date) AS order_day,

   COUNT(*) AS total_orders,

   SUM(total_amount) AS total_revenue

FROM FoodOrders

WHERE order_date >= CURRENT_DATE - INTERVAL 30 DAY

GROUP BY DATE(order_date);
```

Explanation:

- CREATE VIEW: Creates a view named DailyOrderSummary.
- DATE(order_date): Extracts the date from the order date and time.
- COUNT(*): Calculates the total number of food orders per day.

- SUM(total_amount): Calculates the total revenue per day.
- WHERE: Filters orders from the last 30 days.
- GROUP BY: Groups orders by date.

Result: The view displays each date, the total number of orders, and the total revenue for that date over the last 30 days.

5. List 3 good practices you should follow when creating SQL views for analytics dashboards, and for each, give a one-line example related to a Flipkart sales reporting scenario.

1. Use Meaningful Column Names: Choose clear and descriptive column names so that dashboard users can easily understand the data.

Example: In a Flipkart sales reporting view, use total_revenue instead of a confusing name like SUM1.

2. Filter Unnecessary Data: Use the WHERE clause to include only relevant records and improve dashboard performance.

Example: Create a Flipkart sales view that includes only completed orders using WHERE order_status = 'Completed'.

3. Avoid Unnecessary Complexity: Keep views simple and avoid excessive joins or calculations that may slow down dashboard queries.

Example: Create a Flipkart sales view that summarizes total sales by product category using GROUP BY product_category.